

第 18 章 移动应用系统分析与设计

古人云：“学以致用”。一切技术都是因应用而生，为应用服务的，信息技术也不例外。系统分析与设计技术最终要为企业（本书中的“企业”泛指公司、工厂、政府机构、各类组织、各类事业单位等）信息化服务，企业信息化的广阔领域就是系统分析师的用武之地。因此，作为 CIO 的最佳候选人，系统分析师必须掌握有关企业信息化的基础知识，熟悉信息系统建设的基本方法和流程。企业要应对全球化市场竞争的挑战，特别是大型企业要实现跨地区、跨行业、跨所有制、跨国经营的战略目标，要实施技术创新战略、管理创新战略和市场开拓战略，要将企业工作重点转向技术创新、管理创新和制度创新的方向上来，信息化是必然的选择和必要的手段。

本章在介绍移动应用平台基础概念的基础上，从移动应用开发环境、移动应用架构、移动应用开发方式和无代码开发等方面全面介绍了移动应用系统开发知识。

18.1 移动应用平台概述

目前，比较知名且稳定可用的移动开发平台有 EMAS、TMF、FinClip、APICloud、mPaaS、EMOP。其中，mPaaS 和 EMAS 都是阿里巴巴推出的，TMF 是腾讯推出的，EMOP 是京东的产品，它们共同的特点是产品大而全且涉及整个闭环流程，移动开发的全环节面面俱到。FinClip 和 APICloud 属于专注于做移动研发技术平台的公司，产品小而精，有自己独特的特点和优势。

18.1.1 移动端应用开发平台

1. EMAS

EMAS 是阿里巴巴应用研发平台，是面向全端场景（移动 App、H5 应用、小程序、Web 应用、PC 应用等）的一站式应用研发平台。EMAS 基于广泛的云原生技术（Backend as a Service、Serverless、DevOps、低代码等），致力于为企业、开发者提供一站式的应用研发运营管理服务，涵盖开发、测试、运维、运营等应用全生命周期，其示意图如图 18-1 所示。由于 EMAS 和 mPaaS 都来源于阿里巴巴，而且较为类似，就放在一起讲解。mPaaS 是源于支付宝的移动开发平台，为移动开发、测试、运营及运维提供云到端的一站式解决方案，能有效降低技术门槛、减少研发成本、提升开发效率，协助企业快速搭建稳定高质量的移动应用。该产品优势如下：

（1）互联网级视觉体验。阿里巴巴 B-Design 设计体系输出，提供企业级、高品质、差异化视觉界面和 Web/ 移动端组件资产输出，帮助企业构建专属 VI 系统，提供美而简单的数字体验。

（2）现代化应用架构。组件化框架，小程序容器，只需 10min 即可从 0 到 1 搭建超级 App。

（3）阿里巴巴 Mobile DevOps 最佳实践。基于阿里巴巴 Mobile DevOps 最佳实践的生产力和协同平台，实现周粒度的版本迭代和产品交付。

(4) “双十一”应用基础设施。千亿级 API 日调用，并发能力和稳定性经受多年“双十一”洪峰流量考验。

(5) 亿级 App 用户运营工具方法论。超级 App 应用开放框架，互联互通阿里巴巴商业生态，10 亿级流量触达，驱动高质量增长。



图 18-1 EMAS 示意图

2. TMF

TMF 是腾讯移动开发平台，整合了腾讯在移动产品中开发、测试、发布和运营的技术能力，为企业客户提供一站式、覆盖全生命周期的移动端技术平台，其示意图如图 18-2 所示。

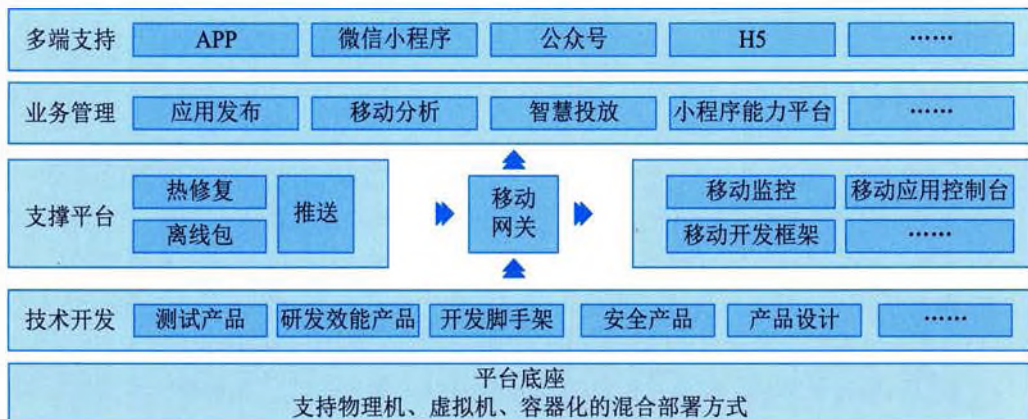


图 18-2 TMF 示意图

3. FinClip

FinClip 是小程序容器技术，相当于把几个巨头的小程序运行能力单独抽离出来，将面向未

来式的开发模式精细化,通过 App 集成 FinClip 小程序 SDK 之后,让 App 能快速具备运行小程序的能力,从而能够实现 App 的解耦,实现模块化开发和动态热更新。该产品优势如下:

(1) 小程序 SDK 保证了业务应用所需要的运行环境。宿主应用如果想与小程序进行数据交互,必须通过 SDK 主动暴露的接口来启动。此外,沙箱环境也能保证小程序的网络通信不被干扰或拦截。

(2) 跨端开发和运行。App (iOS、Android)、计算机 (Windows、Mac、Linux、统信、麒麟)、电视、平板电脑、手表等设备,在集成 FinClip 小程序 SDK 后,都能快速获得运行小程序的能力,真正意义上实现跨端开发和运行。

(3) 自研小程序开发工具。FinClip 自研小程序开发工具 FIDE,任何人都可以通过 FIDE,以“极低的技术投入”将小程序转为独立的 App,并且获得一款可以“安装在手机中的 App”。

(4) 支持灰度发布。FinClip 可按地域、人群、时段等自定义标签进行应用投放,满足企业产品的灰度发布,运营活动 A/B 测试,以及线上版本管理等多场景需求,其优势图如图 18-3 所示。

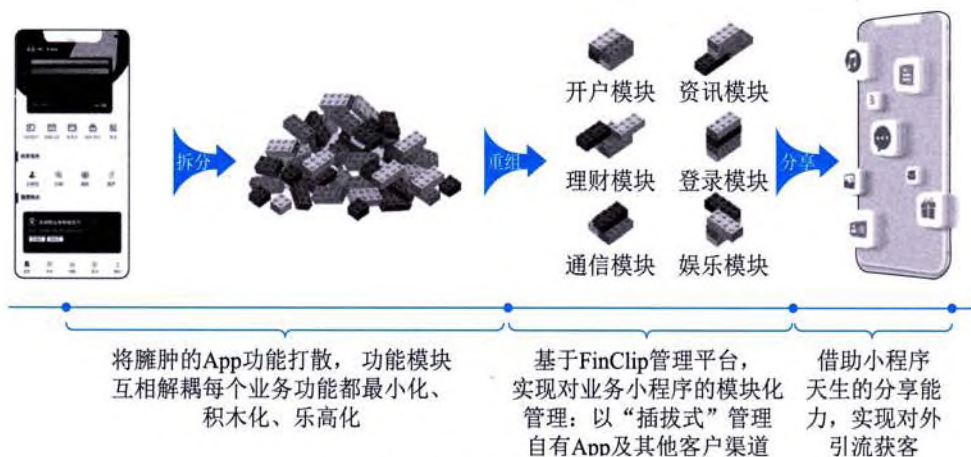


图 18-3 FinClip 优势图

4. APICloud

基于 MADP 构建一款应用时,企业可将 UI 设计、前端开发、后端开发等环节紧密衔接,减少大量重复性工作,并有效提升 30% ~ 60% IT 项目效率。

APICloud 在多端技术领域有 4 个核心能力:

(1) 跨平台引擎。历经近十年的发展,跨平台技术和生态如今趋于稳定,企业和开发者对跨平台技术已形成共识,并大规模地应用到各种业务场景中。

(2) 全流程开发工具。开发工具作为开发者的“灵魂伴侣”,伴随在开发者的整个职业生涯里。市面上有很多流行且成熟的工具可供多端开发进行使用,如 VSCode、Sublime、WebStorm 等。因多端技术 DSL 的个性化,如果能够提供“标准化 + 行业化 + 定制化”的全流程开发工具,包含完备的编码、调试和发布功能,则能让多端开发工作事半功倍,因此诸如微信和支付宝的小程序开发流程中,均提供了符合自己业务特色的 IDE。

(3) 统一编译环境。多终端统一编译环境是一系列代码解析、语法分析、翻译、重构等工具的集合，负责将 DSL 编译为目标编程语言，如将 Less 编译为 CSS。在多端技术领域，多端编译环境负责将 DSL 编译为可在 JS 引擎或者 Web 浏览器中执行的标准 JS 和 HTML 代码，以及编译为符合微信或者支付宝等小程序技术标准的代码。

编译成功的标准 JS 代码运行于跨平台 App 引擎中，实现 App 端功能；小程序代码可直接发布到微信或者支付宝小程序平台，实现小程序端功能；HTML 代码则可作为 Web 端发布。由此达到一套代码可同时部署到 App、小程序、H5 端的目的。

(4) 跨端 JS 架构。在大前端技术领域，当需要解决重复性的问题时，通常会考虑设计一个 DSL (Domain-Specific Language) 或者抽象出一个框架层，专门来解决这些类似的问题。使用 DSL 后并不需要为特定终端编写多套代码，还可以利用宿主语言的抽象能力，确保各终端渲染效果的一致性以及对开发者的友好性。

5. EMOP

企业级移动研发平台结合“京东系”App 研发的经验积累与最佳实践，为移动开发提供一站式解决方案，可以帮助企业构建强大的移动中台，快速创建高质量的 App、各类小程序等移动终端产品，支持企业新业务开展。京东企业级移动研发平台对外开放，它是京东移动中台体系产品化后的“代表作”，涵盖了需求、开发、测试、运维、运营五大领域，可提供企业移动开发的“一站式”解决方案，实现了移动研发全生命周期的技术支撑。其具体有以下特点：

(1) 支持四大开发框架。京东 EMOP 平台同时提供原生、跨端、H5、小程序四大成熟稳定功能强大的开发框架，可结合企业业务特点，协助企业进行深度多端开发及完善，实现一套服务多端输出，仅需一套代码，即可在 Android、iOS、H5、微信小程序等多端使用，有效提高开发协作效率。

(2) 组件化方案实现高效解耦。该平台采用业内先进的模块化、组件化方案，使模块具有高效解耦、并行开发、独立调试、快速构建、灵活集成和动态升级的能力。同时，该平台多年来为内部所有“京东系”App 和小程序的研发提供技术支撑，还积累沉淀了丰富、成熟的技术组件与业务组件，包括网络组件、扫码组件、图片与视频组件和分享组件等，开箱即用，灵活便捷，可根据金融、电商等多场景按需匹配，实现积木式快速搭建 App，高效助力企业降本提效。

(3) 提供 DevOps 持续交付能力。该平台支持完善的 DevOps 流程，提供静态代码扫描、接口管理、持续集成、应用发布、移动配置、热修复等系统，可帮助企业打通各团队流程壁垒，使项目管理人员、开发人员和运维人员可以有效协作，聚焦业务价值的持续交付。

(4) 完善的、系统化的 App 性能监控能力。该平台支持实时展示多维度性能指标与数据趋势分析，全方位掌控线上用户性能问题，可精准定位 App 崩溃问题模块，及时告警突发崩溃状况，第一时间解决崩溃问题；并通过热修复功能，实现无须发版即可快速解决线上问题，用户全程无体验；多管齐下，为应用性能提供全方位护航，给用户带来良好的使用体验。

(5) 提供智能运营分析能力。该平台还支持智能运营分析，基于自动化、大数据实现精准化营销与用户互动，有效降低运营成本，提升用户活跃度与转化率，包括消息推送、视频点/直播、用户反馈、智能客服等系统。

18.1.2 移动端应用部署平台

常见的移动端应用部署平台包括微信小程序、QQ 小程序、抖音小程序、支付宝小程序等，如基于 Android、iOS、嵌入式平台的 App，网页游戏，网站等网页端应用，以及深度集成在手机操作系统中，用户无须下载安装直接在手机上运行，可流畅的体验应用内容的快应用。

1. 小程序部署平台

下面将最为主流的两个小程序部署平台进行对比，即支付宝小程序与微信小程序。

1) 开发基本没有差别

从开发的角度考虑，微信小程序和支付宝小程序几乎没有什么差别。所以很多商家为了在更多的平台推广自己的产品，请第三方公司设计的小程序可以直接在微信和支付宝使用。无论设计哪个平台的小程序，两者通过简单的变化都可以迁移，因此两种小程序的整体构架是没有区别的。但是，从前期开发的成本来说，都是微信平台提供的。所以在这方面支付宝占了好处，而微信取得了先机。

2) 设计逻辑不同

支付宝小程序：功能性会比较复杂，商业属性更强。

微信小程序：在产品逻辑上，微信小程序较为专注简约和直观，只提供了搜索功能及竖排的最近使用微信小程序列表，更加照顾用户体验。

3) 面向人群

微信小程序：目前已经向群众开放，用户与开发者都能通过微信小程序获利。

支付宝小程序：目前还处于公测期间，仅面向企业级用户，即反向拥有企业支付宝账号的开发者开放小程序，暂不向个人开发者开放。

4) 功能区别

微信小程序到目前为止已开放众多服务功能，不论从小程序开发技术本身能够实现的各种功能玩法，还是从微信小程序环境提供的生态构建、产品体系搭建、行业开放类目数量等方面，微信小程序都更胜一筹，可接入的小程序分类也非常丰富。而支付宝基于支付的属性，在社交功能上稍逊一筹，依托蚂蚁金服和芝麻信用，使商业领域延伸出的服务类小程序的金融信用的能力更强，所以在安全支付和金融产品领域则更占优势。

2. App 部署平台

下面对最为主流的 App 部署平台——Android 系统与 iOS 系统进行对比。

1) 开源和闭源

苹果系统基于 UNIX 开发（收费），Android 系统则基于类 UNIX 的 Linux 系统开发（免费）。前者闭源，仅苹果产品使用；后者开源，除内核外开放系统源代码，各厂商可进行深度定制。

2) 硬件资源的使用效率不同

苹果系统基于硬件资源做软件优化，根据系统需求做硬件匹配，使系统充分使用硬件资源，

提升运行效率；而 Android 系统则不同，往往硬件配置得越高，厂商越不做系统的优化，使用效率低。

3) 对应的扩展程序优化不同

苹果系统对其 App 有着严格的审核机制，不符合标准的 App 禁止上架应用商店及提供下载安装；Android 系统的应用软件缺乏监管，多数无法优化到位。

4) 运行机制

两个系统的运行机制不同，iOS 是沙盒运行机制，Android 是虚拟机运行机制。其主要区别在于，iOS 的沙盒运行机制有一个相对规整的分隔控制，相当于把每个应用程序放在自己的沙盒（即一个独立、安全、封闭的存储空间）里进行文件读取，每个沙盒相对独立，每个应用程序运行都在自己的沙盒里进行，相互之间不能互访和干扰。

Android 系统中应用程序的运行则是在虚拟环境中进行的，运行方式是数据先传输到虚拟机，再由虚拟机传递到用户界面（简称 UI），程序之间可以互访，使得内存占用很大、消耗很快，增加垃圾文件的产生，程序运行越多内存占用越快，如此一来，手机就会越用越卡。

5) 后台程序运行

iOS 系统退出应用后无须手动清理后台，系统后台只会保留退出时的界面，实际上相当于已经关闭后台，腾出了内存空间，让手机保持流畅的状态。而 Android 系统打开应用退出后，还是会在后台运行，需要手动关闭，否则后台就一直在运行打开的所有程序，运行程序越多，占用内存就越多，处理器的工作量也越大，很大程度上影响了运行效率。

6) 屏幕响应优先级

两个系统对屏幕反应的优先级不同，iOS 系统最先响应屏幕，用户触摸了屏幕之后，系统就会最优先去处理屏幕显示，即当前手机界面是哪个软件，系统就会优先提供资源让其快速反应，将处理器、运行系统空间进行集中利用，增强用户体验。

Android 系统则是将和屏幕显示相关的处理放在第三层级响应，首先会激活应用、框架，然后才是屏幕。因此，优先级不同导致了 iOS 产品和 Android 产品在操控过程中表现的差异，这也是造成两个系统流畅度不同的因素之一。

7) 系统开放性

iOS 是闭源系统，对其 App 有着严格的审核机制，不符合标准的 App 禁止上架应用商店、禁止下载安装，从源头上杜绝了“流氓”软件和适配性不佳的软件，系统漏洞极少，也难以被第三方软件进行破坏。

Android 是开源系统，对其应用软件缺乏监管，极易受到权限破解及第三程序破坏，尽管底层也加强了安全控制，但总体而言，安全性不如 iOS。

所以，iOS 会比 Android 系统更安全、更流畅；不过由于 iOS 的封闭性，用户限制较多，如不能定制用户界面等；而 Android 系统支持的软件非常丰富，可玩性比较高。

18.2 移动应用开发环境

18.2.1 App开发环境

1. 安卓开发环境

Android App 开发需要的开发环境包括 Java Development Kit (JDK)、Android SDK、Eclipse、Android Studio 等。

1) JDK

JDK 是 Java 的开发工具包,是整个 Java 的核心。JDK 提供了编译、运行 Java 程序所需的各种工具和资源。有了 JDK,Java 开发者就可以编译和运行自己的 Java 程序。JDK 包含的 javac 指令是用来将 .java 文件编译成 .class 文件的,这是开发者需要的编译,而用户(只需要运行程序的人)是不需要的。JDK 包含了 JRE。

2) JRE

JRE 是 Java 的运行环境,它并不是一个开发环境,所以没有包含任何开发工具(如编译器和调试器),只是针对使用 Java 程序的用户。通过 JRE,用户可以运行自己的程序,但是 Java 开发者不能编译自己的代码。JRE 包含了 JVM。

3) JVM

Java 虚拟机 (Java Virtual Machine, JVM) 是一种用于计算设备的规范,它是一个虚构出来的计算机,是通过在实际的计算机上仿真模拟各种计算机功能来实现的。JVM 是 JRE 的一部分,主要负责解释执行字节码文件。开发者写的 Java 源代码经过 Javac.exe 编译成 JVM 能看懂的 .class 字节码文件,JVM 是可运行 .class 字节码文件的虚拟计算机。并且所有平台上的 JVM 向编译器提供相同的接口,而编译器只需要面向虚拟机,生成虚拟机能识别的代码,然后由虚拟机来解释执行。所以,JVM 实现了 Java “一次编译,多处运行”的跨平台特性。

4) Eclipse/IDEA

Eclipse 是免费的,而且可以满足几乎所有的开发需求,经典的计算机应用界面和清晰的目录结构很适合初学者。

IDEA (IntelliJ IDEA) 是一个 Java 语言的开发集成环境,IntelliJ 是业界公认的最优秀的 Java 开发工具,特别是智能代码助手。

Eclipse 与 IDEA 的主要区别如下:

(1) IDEA 没有工作区,IntelliJ IDEA 一次只能处理一个项目,但在 Eclipse 中,可以同时处理多个项目。

(2) IDEA 没有 Perspective,这意味着不能在不同的工作空间布局之间手动切换以执行不同的任务。

(3) IDEA 可以打开多个窗口,在 IntelliJ IDEA 中不能在单个项目中打开多个窗口,但可以将任意数量的编辑器选项卡打开到单独的窗口中。

(4) 代码格式化,IntelliJ IDEA 代码格式化规则与 Eclipse 大致相似,但有一些细微差别。

要导入 Eclipse Code Formatter 设置，需要转到文件 -> 设置 -> 编辑器 -> 代码样式 -> Java，然后选择导入方案 -> Eclipse XML Profile。

5) Android Studio

Android Studio 是谷歌推出的一个 Android 集成开发工具，基于 IntelliJ IDEA。类似 Eclipse ADT，Android Studio 提供了集成的 Android 开发工具用于开发和调试。

2. iOS 开发环境

iOS App 开发需要的开发环境包括 Xcode、iOS SDK、Objective-C 或 Swift 等。其中，Xcode 是苹果公司推出的一款集成开发环境，用于开发 macOS、iOS、iPadOS、watchOS 和 tvOS 应用程序；iOS SDK 是辅助开发 iOS App 的相关文档、API 必需资料、范例和工具的集合；Objective-C 是一种通用、高级、面向对象的编程语言；而 Swift 是苹果公司推出的一种新型编程语言，专门用于开发 iOS 和 macOS 的应用程序。

1) Xcode

Xcode 是运行在操作系统 macOS X 上的集成开发工具（IDE），由 Apple Inc 开发。Xcode 是开发 macOS 和 iOS 应用程序的最快捷的方式。Xcode 具有统一的用户界面设计，编码、测试、调试都在一个简单的窗口内完成。

Xcode 同时也是一种语言。作为一种基于 XML 的语言，Xcode 可以设想各种使用场景。它提供了一种独立于工具的可扩展的方法来描述编译时组件的各方面。

2) iOS SDK

广义上的 SDK（Software Development Kit）是为特定的软件包、软件框架、硬件平台、操作系统等建立应用程序时所使用的开发工具的集合（在 iOS 项目中，SDK 也被称为库）。

在 iOS 开发或 Android 开发中，不可避免会需要使用第三方工具提升产品的开发效率，如用于消息推送的极光、用于第三方支付与登录的支付宝、微信等。但大多数商用产品都不会直接给出源码，在开发 App 时就需要将这些第三方 SDK 集成在项目中。简单来说，SDK 就是提供给开发者的一个工具，使其实现所需的功能。

18.2.2 小程序开发环境

小程序开发需要的技术包括：wxml，与 HTML 网页的编程技术类似；wxss，大部分和原先的 CSS 基本相同；js；json，通过 json 控制上下菜单栏、主次的页面展示顺序。

此外，小程序运行在多种平台上：iOS/iPadOS 微信客户端、Android 微信客户端、Windows PC 微信客户端、Mac 微信客户端、小程序硬件框架和用于调试的微信开发者工具等。

1. 微信开发者工具

微信开发者工具基于 nw.js，使用 node.js、chromium 以及系统 API 来实现底层模块，使用 React、Redux 等前端技术框架来搭建用户交互层，实现同一套代码跨 Mac 和 Windows 平台使用。

WXML（WeiXin Markup Language）是小程序框架设计的一套标签语言，用于构建出页面

的结构。小程序渲染层的运行环境是一个 WebView，而 WebView 无法直接理解 WXML 标签，所以需要经过编译。

微信开发者工具内置了一个二进制的 WXML 编译器，这个编译器接受 WXML 代码文件列表，处理完成之后输出 JavaScript 代码，这段代码是各个页面的结构生成函数。

WXSS (WeiXin Style Sheets) 是一套样式语言，用来决定 WXML 的组件应该怎么显示。为了适应广大的前端开发者，WXSS 具有 CSS 的大部分特性。同时为了更适合开发微信小程序，WXSS 对 CSS 进行了扩充以及修改。

微信客户端在运行小程序的逻辑层时只需要加载一个 JS 文件（即 app-service.js），而小程序框架允许开发者将 JavaScript 代码写在不同的文件中，所以在代码上传之前，微信开发者工具会对开发者的 JS 文件做一些预处理，包括 ES6 转 ES5 和代码压缩（开发者可以选择关闭预处理操作），在服务器编译过程将每个 JS 文件的内容分别包裹在 define 域中，再按一定的顺序合并成 app-service.js。其中，对于页面 JS 和 app.js 需要主动 require。

2. 模拟器

小程序模拟器模拟小程序在微信客户端的逻辑和界面表现，方便开发者实时查看代码效果。由于系统差异以及微信客户端特有的一些交互流程，少部分的 API 无法在模拟器上进行模拟，但对于绝大部分的 API 均能够在模拟器上呈现出正确的状态。同时，微信开发者工具提供多种机型尺寸以及自定义机型尺寸功能，方便开发者进行界面样式的机型适配。

18.2.3 网页应用开发环境

移动端网页应用的开发环境包括如下三种：

- (1) HTML5 开发工具，如 DevExtreme。
- (2) 移动设备开发技术。设计在移动设备上运行良好的网站所需的一些主要技术，如跨浏览器兼容性、移动设备设计等。
- (3) 移动端开发技术，如 Native App、Hybrid App 等。

18.3 移动应用架构

应用架构是指按照某种规范和约束将业务能力进行拆分，并由不同应用（系统）承接的结构载体，从而能够实现将拆分后的应用以一种规律有序的方式进行连接并创造业务活动。应用架构分为以下两个不同的层次：企业级的应用架构和系统级的应用架构。

企业级的应用架构起到了统一规划、承上启下的作用，向上承接了企业战略发展方向和业务模式，向下规划和指导企业各个 IT 系统的定位和功能。而系统级的应用架构则是在企业级应用架构的指导下，对单个系统进行设计和实现。应用架构设计可以帮助开发人员更好地理解 and 把握软件系统，从而更好地满足用户需求，提高软件质量，降低开发成本。

移动应用的架构发展历史可以大致分为以下几个阶段。

1. 单体应用阶段

早期的移动应用开发采用单体应用架构，即所有功能模块都集成在一个应用程序中，是一种将所有业务场景的表示层、业务逻辑层和数据访问层放在一个工程中的应用架构模式。例如，开发一个进销存系统，可以将项目打包成 war 包并部署到服务器上，这样的 war 包涵盖了很多模块。

单体应用架构模式的优点是：开发简单、部署简单、运维简单、性能稳定。但是，随着业务的发展，单体应用会变得越来越复杂，代码量也会越来越大，这样会导致开发效率低下、维护成本高。

2. MVC 架构阶段

随着移动应用市场的快速发展，MVC（Model-View-Controller）架构开始逐渐被应用到移动应用开发中。该架构将应用程序分为三层：模型层、视图层和控制器层，实现了代码解耦和代码复用，提高了应用的可扩展性和可维护性。同时，该架构也引入了一些新的问题，如控制器层可能会变得庞大而难以管理。

MVC 架构示意图如图 18-4 所示。下面举一个常见的例子来具体说明 MVC 各自的作用。例如，小时候玩的卡带式游戏机，Control 是主机，一般来说买一个主机就行了，只要不坏，就能一直玩这一类的游戏。View 则是电视机和游戏手柄，电视机可以独立工作，不管输入的是电视信号、影碟机信号还是游戏机信号，它只管显示，而且电视机决定了我们看到的效果是怎样的，如果想要一个尺寸更大的或者彩色的显示效果，只需要买个相应的电视机即可，手柄也是可以换的。Model 则是游戏卡带，它决定了玩的是什么游戏，是魂斗罗还是超级玛丽，而且游戏机主机和电视机生产厂家永远也不知道在上面有可能会运行什么样的游戏。卡带中可能会有游戏代码和存储单元，都根据游戏的需要而设计。

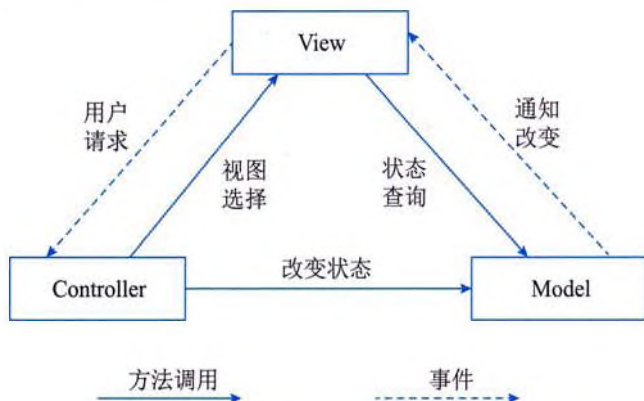


图 18-4 MVC 架构示意图

MVC 主要的框架有 Spring、Struts ZF、.NET。以 SpringMVC 为例来解释 MVC 在 Spring 的工作流程中如何体现的。

SpringMVC 是一个 MVC 的开源框架，SpringMVC=struts2+spring，相当于 Struts2 加上

Spring 的整合。但是这里有一个疑惑，SpringMVC 和 Spring 是什么样的关系呢？有一个很好的解释是：SpringMVC 是 Spring 的一个后续产品，其实就是 Spring 在原有基础上，又提供了 Web 应用的 MVC 模块，可以简单地把 SpringMVC 理解为 Spring 的一个模块（类似 AOP、IOC 这样的模块），我们经常说 SpringMVC 和 spring 无缝集成，其实 SpringMVC 就是 Spring 的一个子模块，所以不需要同 Spring 进行整合。

图 18-5 将前端的视图用 jsp 来代替，DAO 是 MVC 外的另一个层，即数据持久层，主要进行对数据库的操作。

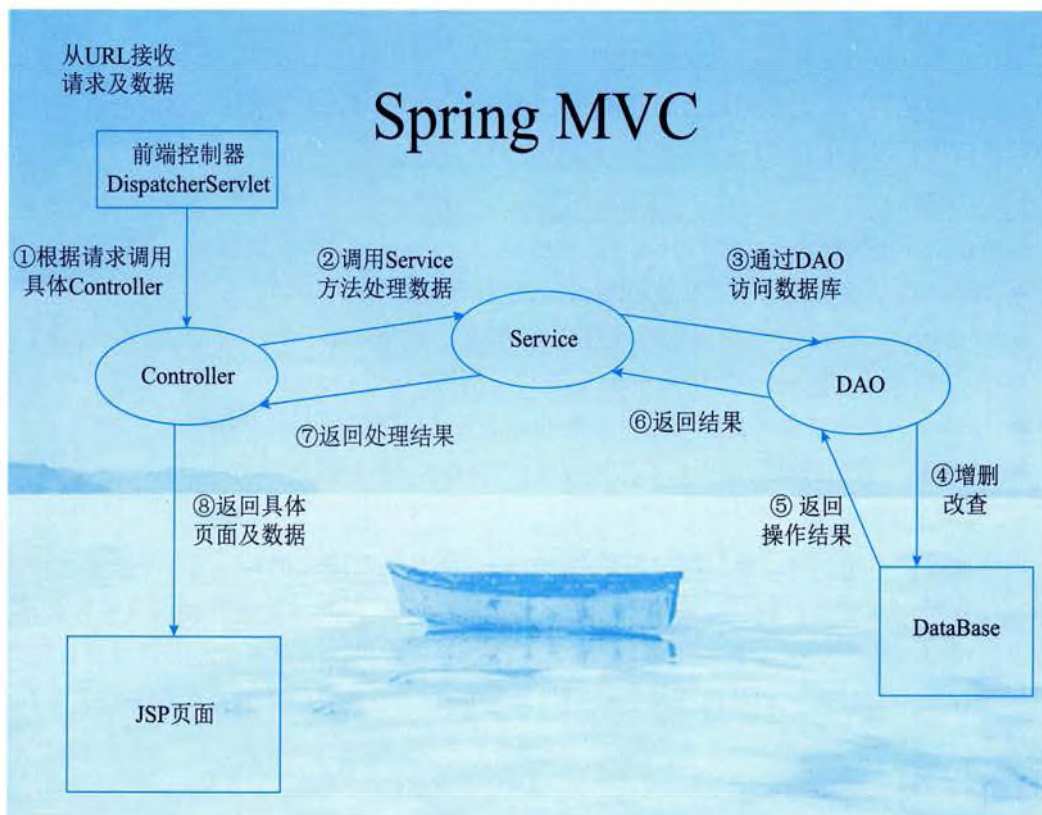


图 18-5 SpringMVC 示意图

SpringMVC 的工作流程如下：

- (1) 用户发起请求到前端控制器（DispatcherServlet）。
- (2) 前端控制器请求处理器映射器（HandlerMapping）去查找处理器（Handle）：通过 xml 配置或者注解进行查找。
- (3) 找到处理器映射器（HandlerMapping）后，向前端控制器返回执行链（HandlerExecutionChain）。
- (4) 前端控制器（DispatcherServlet）调用处理器适配器（HandlerAdapter）去执行处理器（Handler）。
- (5) 处理器适配器去执行 Handler。

- (6) Handler 执行完给处理器适配器返回 ModelAndView。
- (7) 处理器适配器向前端控制器返回 ModelAndView。
- (8) 前端控制器请求视图解析器 (ViewResolver) 去进行视图解析。
- (9) 视图解析器向前端控制器返回 View。
- (10) 前端控制器对视图进行渲染。
- (11) 前端控制器向用户响应结果。

下面是一个 MVC 架构案例。

1) 项目背景

微信是一款由中国腾讯公司开发的即时通信软件，于 2011 年发布。它在移动设备上提供了各种功能，包括聊天、语音通话、朋友圈、支付等。微信是一个非常成功的移动应用，目前在全球范围内拥有数十亿的用户。

2) 技术栈

微信移动应用的技术栈主要包括以下几方面：

- Java。微信移动应用的后端使用Java语言进行开发。
- Objective-C/Swift。微信移动应用的iOS客户端使用Objective-C或Swift进行开发。
- Java/Kotlin。微信移动应用的Android客户端使用Java或Kotlin进行开发。
- Node.js。微信支付和其他一些服务使用Node.js进行开发。
- MySQL。微信移动应用的后端数据存储使用MySQL数据库。

3) 功能模块

微信移动应用的功能模块包括：用户可以在微信中发送和接收文本、图片、语音、视频等消息，并支持一对一和群聊；用户可以添加、删除和管理联系人，并查看联系人的详细信息；用户可以在朋友圈中发布文字、图片、视频等内容，并查看其他用户的动态、点赞和评论；用户可以在微信中进行支付和转账等操作；微信还提供了许多其他的功能，如扫一扫、公众号、小程序等。

4) MVC 架构的具体使用

微信移动应用使用了 MVC 架构，将各个功能模块分为模型层 (Model)、视图层 (View) 和控制器层 (Controller)。以聊天模块为例，具体使用如下：

- 模型层定义聊天消息、聊天记录等数据。在微信中，消息的数据结构包括消息类型、发送者、接收者、内容等。
- 视图层展示聊天界面。在微信中，聊天界面展示了所有聊天消息，包括文本、图片、语音、视频等。
- 控制器层实现消息的发送和接收，聊天记录的存储等逻辑。在微信中，聊天控制器负责发送和接收消息，并将消息存储到本地数据库中。

通过 MVC 架构，微信实现了模块化开发，各个模块之间松散耦合，便于后续扩展和维护。例如，在聊天模块中，如果需要增加一个新的消息类型，只需要修改模型层的数据结构和控制

器层的发送和接收逻辑，而不需要修改视图层的界面展示逻辑，这使得开发更加灵活和高效。

5) 总结

微信移动应用采用了 MVC 架构，将各个功能模块分为模型层、视图层和控制器层。通过 MVC 架构，微信实现了模块化开发，各个模块之间松散耦合，便于后续扩展和维护。微信移动应用的技术栈主要包括 Java、Objective-C/Swift、Java/Kotlin、Node.js 和 MySQL 等。微信移动应用的功能模块包括聊天模块、联系人模块、朋友圈模块、支付模块和其他模块。微信移动应用是一款非常成功的移动应用，为用户提供了丰富的功能和优秀的用户体验。

3. MVP 阶段

在 MVC 的设计中，将 View 和 Controller 的代码放到一起，这样会导致 Activity/Fragment 中的代码越来越多，所以开始分离 VC 中的代码。

将 Activity/Fragment 中复杂的逻辑处理移至另外的一个类（Presenter）中，此时，Activity 其实就是 MVP 模式中的 View，负责 UI 元素的初始化，建立 UI 元素与 Presenter 的关联（Listener 之类），同时自己也会处理一些简单的逻辑，复杂的逻辑交由 Presenter 处理。

模型（Model）层：用于封装与应用程序的业务逻辑相关的数据以及对数据的处理方法。模型层有对数据直接访问的权利，如访问数据库、获取网络数据。Model 不依赖 View 和 Presenter，也就是说，Model 不关心它是如何显示或者如何被操作。

视图（View）层：负责显示数据，与用户进行交互。持有逻辑层的引用。在 Android 中体现为 Activity/Fragment。

逻辑（Presenter）层：负责逻辑处理，通过模型层获取数据，回显到 UI 层，响应用户的行为。

MVP 架构是一种软件设计模式，它将应用程序分为三个层次，即 Model、View 和 Presenter。Model 负责处理数据和业务逻辑，View 负责显示用户界面，Presenter 负责协调 Model 和 View 之间的交互。MVP 架构的目的是降低 View 和 Model 之间的耦合，提高代码的可测试性和可维护性。MVP 架构示意图如图 18-6 所示。

下面是一个 MVP 架构案例。

1) 项目背景

美食家是一款以美食为主题的移动应用程序，该应用程序旨在为用户提供美食推荐、餐厅预订、菜谱分享等服务。在该应用程序中，MVP（Model-View-Presenter）架构被用来实现代码的模块化、可复用性和可测试性。

2) 架构设计

在 MVP 架构中，应用程序被分为三个组件：模型（Model）、视图（View）和逻辑（Presenter）。在美食家应用程序中，这三个组件的具体实现如下：

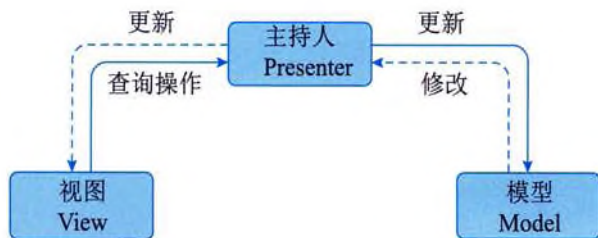


图 18-6 MVP 架构示意图

模型层：美食家应用程序的模型层被设计为一个数据仓库，负责处理数据逻辑和管理数据源。具体来说，该层包含了一个 SQLite 数据库和一个 Retrofit 库。SQLite 数据库被用来存储应用程序的本地数据，如用户信息、菜谱信息等。Retrofit 库被用来处理网络请求和 JSON 数据的解析。

视图层：美食家应用程序的视图层被设计为一个 UI 层，负责展示数据和接收用户的操作。具体来说，该层包含了各种 Android 的 UI 组件，如 TextView、EditText、ListView 等。这些 UI 组件被用来展示美食信息、菜谱信息、用户信息等。

逻辑层：美食家应用程序的逻辑层被设计为一个业务逻辑层，负责处理应用程序的业务逻辑。具体来说，该层通过调用模型层和视图层的 API 来实现业务逻辑。该层使用了 RxJava 和 Dagger2 框架，以实现响应式编程和依赖注入。

3) 开发思路

美食家应用程序的开发思路是将整个应用程序划分为多个功能模块，并将每个模块的代码分别实现在 MVP 架构的三个组件中。在每个组件中，使用接口来定义组件之间的交互，以实现代码的模块化和可复用性。同时，使用依赖注入来管理组件之间的依赖关系，以实现代码的可测试性。

4) 功能模块

美食家应用程序包含以下功能模块：

- (1) 用户注册、登录和个人信息管理。该模块包括用户注册、登录、修改个人信息等功能。
- (2) 美食推荐和搜索。该模块包括美食推荐、搜索美食等功能。
- (3) 餐厅预订和评论。该模块包括餐厅预订、取消预订、餐厅评论等功能。
- (4) 菜谱分享和收藏。该模块包括菜谱分享、收藏菜谱等功能。
- (5) 美食地图和导航。该模块包括美食地图、导航等功能。

以上模块分别被实现在 MVP 架构的三个组件中，以实现代码的模块化、可复用性和可测试性。

5) 总结

美食家应用程序是一款使用了 MVP 架构的移动应用程序，该应用程序采用了模块化、可复用性和可测试性的开发思路，并实现了多个功能模块。同时，该应用程序的架构设计具有良好的可扩展性和可维护性，以满足应用程序的长期需求。

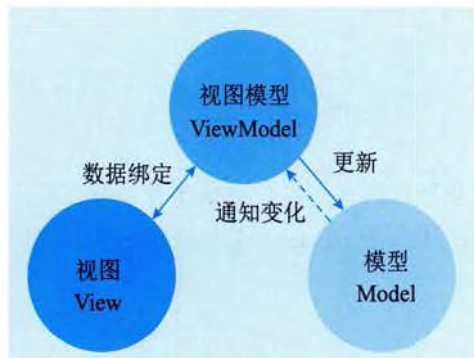


图 18-7 MVVM 架构示意图

4. MVVM 架构阶段

MVVM (Model-View-ViewModel) 架构是在 MVC 架构的基础上发展起来的。MVVM 架构示意图如图 18-7 所示。该架构通过引入 ViewModel 层，进一步解耦了视图和模型，使每个组件的职责更加清晰明确。另外，MVVM 架构还引入了数据绑定和响应式编程的思想，使得应用程序能够更加灵活地

响应用户输入。MVC → MVP → MVVM 这几个软件设计模式是一步一步演化发展的，MVVM 是 MVP 的进一步发展与规范，MVP 隔离了 MVC 中的 M 与 V 的直接联系后，靠 Presenter 来中转，所以使用 MVP 时 P 是直接调用 View 的接口来实现对视图的操作的，这个 View 接口的东西一般来说是 showData、showLoading 等。M 与 V 已经隔离了，方便测试，但代码还不够优雅简洁，所以 MVVM 就弥补了这些缺陷。

下面介绍一个 MVVM 架构案例。

VUE.JS 就是基于 MVVM 模式实现出来的一套框架（基于 MVVM 架构的还包括：微信小程序、AngularJS、React）。

VUE 是当前热门的前端框架之一，大部分的网站开发都将 VUE 作为首选。

VUE 中使用了 MVVM 的思想，如图 18-8 所示，其中架构的组件如下：

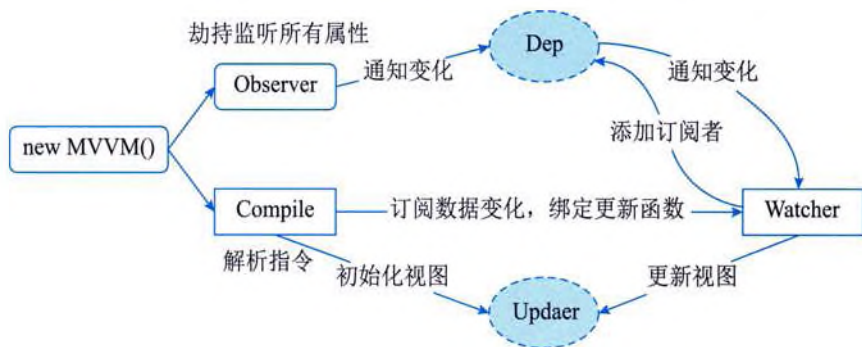


图 18-8 VUE 中的 MVVM 思想

(1) `Observer`。数据监听器，能够对数据对象的所有属性进行监听，如有变动可拿到最新值并通知订阅者，内部采用 SE5 的 `Object.defineProperty` 的 `getter` 和 `setter` 来实现。

(2) `Compile`。指令解析器（`v-for`，`v-if`，...），它的作用是对每个元素节点的指令进行扫描和解析，根据指令模板替换数据，以及绑定相应的更新函数。

(3) `Watcher`。订阅者，作为连接 `Observer` 和 `Compile` 的桥梁，能够订阅并收到每个属性变动的通知，执行指令绑定的相应回调函数。

(4) `Dep`。消息订阅器，内部维护了一个数组，用来收集订阅者（`Watcher`），数据变动触发 `notify` 函数，再调用订阅者的 `update` 方法。

5. 微服务架构阶段

近年来，随着移动应用规模的不断扩大和功能的复杂化，微服务架构逐渐成为了移动应用开发的趋势。该架构将应用程序分解为多个独立的服务，每个服务都可以独立部署和扩展，从而提高了应用的伸缩性和容错性。同时，微服务架构也带来了一些新的挑战，例如，服务间通信和数据一致性问题需要得到更好的解决。

微服务本身的架构也经过很多次的演变，微服务架构演进历程主要经历了以下几个阶段。

1) 基于服务实现模块化

早期的服务化架构主要是将系统拆分为不同的模块并通过服务实现模块化，但服务本质上

仍然是单体应用。

2) 基于 SOA 的服务拆分

随着 SOA 的兴起，微服务架构概念得以深入发展。服务化架构开始通过服务拆分来实现更小的服务颗粒度。同时，服务治理和服务注册中心等基础设施也得以快速发展。

3) 基于容器和服务框架

随着容器技术的成熟，微服务架构开始基于容器进行部署和管理，同时一批开源的服务框架和基础设施也应运而生，如 SpringCloud、Netflix OSS 等。SpringCloud 的微服务示意图如图 18-9 所示。

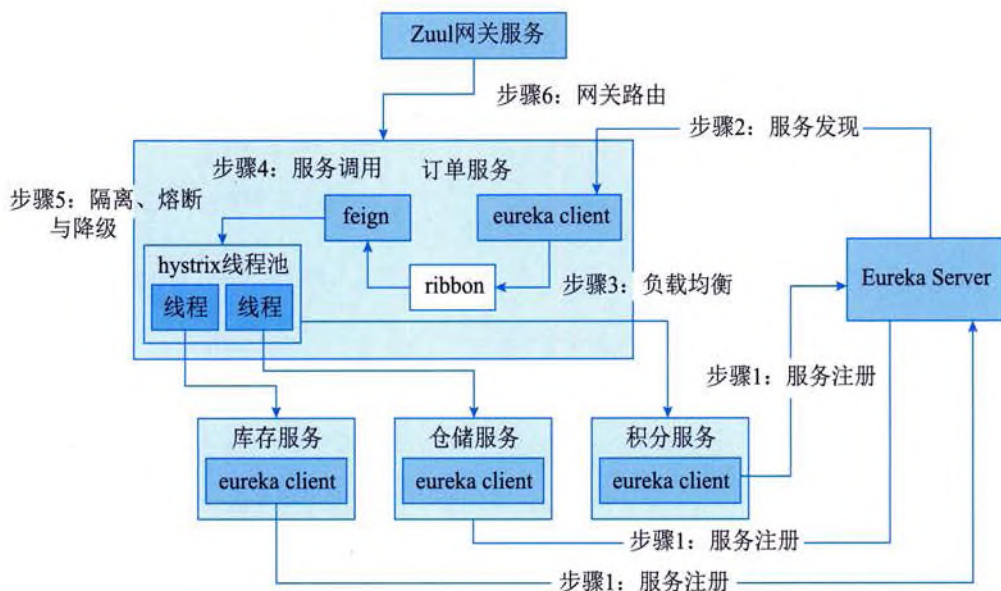


图 18-9 SpringCloud 的微服务示意图

4) 基于云原生的微服务架构

在云原生技术的推动下，微服务架构已经从传统的部署方式转向了基于云平台的统一部署和运维。新的云原生技术如 Kubernetes 等，提供了更高效的容器编排、负载均衡和自动化部署等能力，大大简化了微服务架构的部署和管理。

微服务是一种基于轻量级、面向服务、可独立部署和升级的分布式架构模式，随着企业业务和系统的复杂度不断提升和变化，微服务架构也在不断演进和完善。通过技术的创新、架构思想的转变，微服务架构已经成为各大企业建立基于云原生的分布式系统的首选架构。

6. 微服务架构案例

一个使用了微服务架构的知名案例是 Netflix。Netflix 是一个全球知名的在线流媒体平台，采用了微服务架构来构建其庞大的应用系统。以下是 Netflix 微服务架构的不同功能模块以及它们之间的关系。

(1) 用户管理服务 (User Management Service)。用户管理服务负责处理用户注册、登录、

账号管理等功能。它处理用户身份验证和授权,并维护用户的个人信息。这个服务与其他服务进行通信,以验证用户的访问权限,并确保用户信息的一致性。

(2) 视频流服务 (Video Streaming Service)。视频流服务是 Netflix 的核心服务之一,负责流媒体内容的存储、编码、传输和分发。它处理视频的上传、编码、储存和转码,并通过合适的协议将视频内容传输给用户。这个服务与用户管理服务进行交互,以确保只有经过授权的用户可以访问视频内容。

(3) 推荐系统服务 (Recommendation System Service)。推荐系统服务分析用户的观看历史、评分、喜好等数据,并根据个性化算法生成推荐内容。它与用户管理服务和视频流服务进行通信,获取用户信息和视频内容数据,从而生成针对每个用户的个性化推荐列表。

(4) 支付服务 (Payment Service)。支付服务负责处理用户的订阅和付款操作。它与用户管理服务进行交互,验证用户的付款信息并处理订阅变更和付款事务。支付服务还与其他外部支付网关和服务进行集成,以完成支付操作。

(5) 配置管理服务 (Configuration Management Service)。配置管理服务负责管理整个微服务架构中的配置信息。它存储各个微服务的配置参数,如数据库连接信息、API 密钥等。其他服务可以通过调用配置管理服务来获取其所需的配置信息。

这些不同的功能模块通过轻量级的通信机制 (通常是基于 HTTP 的 RESTful API) 进行交互。每个微服务运行在自己的进程中,独立部署和扩展。微服务之间通过 API 调用和消息传递进行通信,并通过服务注册和发现机制来实现集中管理。

总的来说,Netflix 采用微服务架构将其庞大的应用系统划分为多个独立的服务,每个服务专注于一个特定的功能领域。这种架构使得 Netflix 能够实现高可用性、可扩展性和灵活性,并且不同功能模块之间的解耦性使得各个模块可以独立开发、部署和扩展。

18.4 移动应用开发

应用开发技术是指使用不同的编程语言和工具来创建和维护各种应用程序的技术。应用开发技术可以分为原生应用开发、混合应用开发和 Web 应用开发等类型,根据不同的平台和需求选择不同的技术方案。

原生应用开发是指针对特定的操作系统 (如 iOS 或 Android) 使用原生编程语言 (如 Objective-C 或 Java) 来开发的应用程序,它们可以充分利用设备的硬件和软件功能,提供最佳的用户体验和性能,但是开发难度高,成本高,需要为不同的平台分别开发。

混合应用开发是指使用网页技术 (如 HTML5、CSS3、JavaScript) 来开发的应用程序,它们可以在不同的平台上运行,具有跨平台的优势,同时也可以通过调用原生 API 来实现一些特定的功能,提高用户体验,但是性能可能不如原生应用,页面交互效果也有限。

Web 应用开发是指使用网页技术来开发的网站或网页,它们不需要下载安装,只需要通过浏览器访问即可,具有便捷性和易维护性的优势,但是用户体验可能不足,页面跳转迟钝或卡顿,功能也受到浏览器的限制。

18.4.1 开发语言

1. 常见开发 App 语言

常见的开发 App 的语言包括：

(1) 原生 App (Native App)。Android 中使用 Java/Kotlin 开发，iOS 中使用 Objective-C/Swift 开发。其优势是效率高；不足是费用高。

(2) Web App。使用 HTML/CSS/JS 编写，运行在浏览器中，封装为 App 安装文件，在应用商店中上架。其优势为一套代码可以运行在两个平台；不足为执行效率不如原生 App。

(3) H5。使用 HTML/CSS/JS 编写，运行在浏览器中，不需要应用商店中上架，也没有图标，通过二维码在朋友圈中快速分享。其优势为一套代码可以运行在两个平台；不足为执行效率不如原生 App。

(4) 混合 / 混编 App (Hybrid App)。使用 HTML/CSS/JS/Native.js(Java/OC=>JS)。其优势为一套代码可以运行在两个平台，还可以调用系统底层原生功能；不足为执行效率不如原生 App。

(5) 桥接 App (Bridged App)。桥接 App 只使用 JS 编写，执行环境会把 JS 编译为 Java/OC，在手机中运行时就是纯原生代码。其优势为一套代码可以运行在两个平台，执行效率高；不足为两个平台不能做到完全一样。

(6) Flutter App。使用 Dart 语言开发，直接在 Android/iOS 手机中的显卡里进行绘图。其优势为一套代码可以运行在两个平台，执行效率高，效果完全一样；不足为全新的语言，Google 开发的进度比较慢。

18.4.2 小程序开发方式

以微信小程序举例，即微信小程序原生开发、混合开发和第三方组件 (SDK) 开发。其中，微信小程序原生开发通常是指使用微信公众号平台提供的开发工具来进行开发，混合开发是指将原生开发与第三方组件开发结合起来，而第三方组件开发则是指使用第三方组件来开发微信小程序。微信小程序的开发技术主要包括 wxml、wxss、js 和 json。其中，wxml 是小程序的模板语言，类似于 HTML；wxss 是小程序的样式语言，类似于 CSS；js 是小程序的逻辑语言，类似于 JavaScript；json 是小程序的配置文件，用于对当前小程序进行全局配置。此外，微信小程序借鉴了很多前端开发的技术理念，它用 React 实现了“视觉组件”，用 CMD 的 require 作为面向对象的 JavaScript，用 Vue 实现了标签式逻辑与数据绑定。

小程序开发方式有多种，主要有以下几种。

1. 基于微信小程序框架开发

使用微信提供的 API 进行小程序开发，新手友好，上手快，可以节省大量开发时间；小程序开发框架的目标是通过尽可能简单、高效的方式让开发者可以在微信中开发具有原生 App 体验的服务。微信团队为小程序提供的框架命名为 MINA。MINA 框架通过封装微信客户端提供的文件系统、网络通信、任务管理、数据安全等基础功能，对上层提供一整套 JavaScript API，让开发者方便地使用微信客户端提供的各种基础功能与能力，快速构建应用。

2. 原生开发方式

基于原生语言（如 Java、Object-C）开发，使用微信官方平台提供的 SDK 进行开发，能够实现更多高级功能，可以拥有更好的性能。

3. 网页开发方式

基于网页语言（如 HTML、CSS、JavaScript）开发，使用微信官方提供的 API 进行小程序开发，可以节省大量开发时间，小程序可以兼容不同平台，使用更方便。

4. 跨平台开发方式

基于跨平台语言（如 React Native、Weex）开发，可以节省大量开发时间，实现跨平台的功能，使用更方便。

5. 云开发方式

基于微信小程序云开发，可以节省大量开发时间，实现快速搭建开发小程序。微信云开发是微信团队联合腾讯云推出的专业小程序开发服务。开发者可以使用云开发快速开发小程序、小游戏、公众号网页等，并且原生打通微信开放能力。开发者无须搭建服务器，可免鉴权直接使用平台提供的 API 进行业务开发。只需使用平台提供的各项能力，即可快速开发业务。无须管理证书、签名、秘钥，直接调用微信 API。复用微信私有协议及链路，保证业务安全性。支持环境共享，一个后端环境可开发多个小程序、公众号、网页等，便捷复用业务代码与数据。支持按量计费模式，后端资源根据业务流量自动扩容，先使用后付费，无须支付闲置成本。

18.4.3 原生App开发方式

App 开发技术可以分为三大类：原生应用（Native Application, Native App）、Web 应用（Web Application, Web App）和混合应用（Hybrid Application, Hybrid App）。其中，原生应用是专门为特定手机平台开发的应用程序，无法在其他平台运行。常用的开发技术有 Swift、OC、Java 等。Web App 多数使用 HTML 或 HTML5、CSS3、JavaScript 技术做 UI 布局，使其在网站页面上实现传统的 C/S 架构软件功能，服务端技术用 Java、PHP、ASP 等。混合应用是介于原生应用和 Web App 之间的一种应用类型，它既可以像原生应用一样访问设备 API，又可以像 Web App 一样使用 HTML5、CSS 和 JavaScript 等 Web 技术进行开发。

开发 Native App 需要根据运行的手机系统采用不同的开发语言，开发安卓 App 需要熟悉 Java 语言，对于 Android 环境和机制要熟悉，学会搭建 Android 开发环境。此外，一些 App 的某些功能还可能涉及算法，需要有一定的数学基础，了解数据结构。

1. Android 开发

Android 开发中使用的主要编程语言是 Java，Java 是核心。然而，学会 Java 还远远不够，还需要掌握一些 UI 设计、第三方类库调用、网络和数据库等方面的知识。此外还需具备以下条件：

（1）需求完整。需要制定完整的需求文档、功能文档、流程图和时序图，用于早期需求规划和交互设计。

(2) 交互设计。UI 设计需设计出原型图和 App 基本完善的交互设计效果，然后根据这些设计一个完整的 UI 界面。其中，交互设计需要懂得很多人机操作的技巧和经验，掌握 Axure 等交互工具的使用，UI 设计需要掌握 Photoshop 和 Illustrator。

(3) 使用开发环境。例如，App 开发的 ADT。其基础是必须掌握 Java 语言，熟悉 Android 环境和机制，涉及的知识面较广，请开发人员根据项目自行学习相关知识。

2. iOS 开发

iOS 开发需要掌握以下方面的知识：

(1) 学习 Objective-C 和 Swift 编程语言。掌握苹果 App 开发的基本原理，以及 Objective-C 和 Swift 语言的语法和特性。

(2) 使用 Xcode 进行苹果 App 软件开发。包括如何进行调试，如何使用苹果提供的模板，以及如何使用 Storyboard 创建 UI 界面等。

(3) 使用苹果的 Cocoa Touch 框架。掌握 iOS 应用开发的基本原理，以及如何使用 Cocoa Touch 框架进行苹果 App 开发。

(4) 学习如何使用苹果的硬件编程接口。如如何使用相机、蓝牙等硬件编程接口开发苹果 App。

(5) 使用 CoreData 进行数据库管理。掌握 CoreData 的基本原理，以及如何使用 CoreData 进行苹果 App 开发。

(6) 学习如何使用 WebKit 框架进行苹果 App 开发。掌握 WebKit 的基本原理，以及如何使用 WebKit 框架进行苹果 App 开发。

此外，由于 iOS 系统的封闭性，开发过程还需要注意以下几方面：

(1) 苹果 App 软件开发过程中应该重视安全。例如，注意编码规范，使用高安全级别的编码，以及保护用户的隐私。

(2) 苹果软件开发过程中应该注意性能。例如，尽量减少 App 的内存使用，以及尽量减少 App 的网络使用。

(3) iOS 软件开发过程中应该注意可维护性。例如，尽量使用可维护的代码，以及尽量使用可维护的架构。

(4) 苹果应用开发过程中应该注意测试。例如，尽量进行功能测试，以及尽量进行性能测试。

(5) iOS 开发过程中应该注意发布。例如，准备发布时需要收集必要的信息，以及需要准备发布需要的证书等。

(6) iOS App 软件开发过程中应该注意版本更新。例如，需要定期更新 App 的版本，以及需要对 App 的新版本进行测试等。

3. 安卓开发案例

1) 项目背景

滴滴出行是一款为用户提供出租车、快车、拼车等多种出行方式的应用。它于 2012 年由中国的一家公司推出，目前在全球范围内拥有数亿用户。滴滴出行的目标是通过技术创新，提高

城市出行效率,改善城市居民的出行体验。

2) 技术栈

Java/Kotlin: 滴滴出行的安卓客户端使用 Java 或 Kotlin 进行开发; **Node.js:** 滴滴出行的后端服务使用 Node.js 进行开发; **React:** 滴滴出行的前端页面使用 React 进行开发; **MySQL:** 滴滴出行的数据库使用 MySQL 进行存储。

3) 功能模块

滴滴出行包含以下 5 个功能模块。车辆服务模块: 用户可以通过滴滴出行预订出租车、快车、拼车等服务; 顺风车模块: 用户可以通过滴滴出行找到同路人, 共享出行, 减少交通拥堵; 顺路送模块: 用户可以通过滴滴出行找到同路人, 共享快递、外卖等物品的配送服务; 代驾模块: 用户可以通过滴滴出行预订代驾服务, 解决酒后驾车的问题; 停车模块: 用户可以通过滴滴出行查找附近的停车位, 实现停车便利。

4) 架构思路

滴滴出行采用了微服务架构, 将各个功能模块分为独立的服务, 通过 API 进行通信。这样做的好处是各个服务之间松散耦合, 便于扩展和维护。

在安卓客户端的开发中, 滴滴出行采用了 MVP 架构, 将各个功能模块分为模型层、视图层和控制器层。通过 MVP 架构, 滴滴出行实现了模块化开发, 各个模块之间松散耦合, 便于后续扩展和维护。

在安卓客户端的开发中, 滴滴出行采用了敏捷开发的方法。敏捷开发是一种以迭代、循序渐进的方式进行开发的方法。在滴滴出行的开发中, 每个开发周期一般为 2 周, 每个开发周期结束后会进行一次迭代。在每个迭代中, 会完成一些小的功能, 或者对已有的功能进行优化和改进。这种方法可以让开发团队更加灵活地应对需求变化, 同时也可以让产品更快上线, 提高产品的迭代速度和质量。

5) 总结

滴滴出行是一款为用户提供出租车、快车、拼车等多种出行方式的应用。它采用了微服务架构, 将各个功能模块分为独立的服务, 通过 API 进行通信。在安卓客户端的开发中, 滴滴出行采用了 MVP 架构和敏捷开发的方法。这种架构和方法可以让开发团队更加灵活地应对需求变化, 同时也可以让产品更快上线, 提高产品的迭代速度和质量。

4. iOSApp 开发案例

1) 项目背景

知乎是一家基于互联网的知识分享平台, 于 2011 年成立。知乎的使命是为用户提供一个有价值、有深度、有意思的知识分享平台。知乎的 iOS 客户端是该平台的重要组成部分, 为用户提供了丰富的功能和良好的用户体验。

2) 技术栈

知乎 iOS 客户端的技术栈主要包括以下几方面。Swift/Objective-C: 知乎 iOS 客户端使用 Swift 或 Objective-C 进行开发; MVVM 架构: 知乎 iOS 客户端采用了 MVVM 架构, 将各个功

能模块分为模型层、视图层和视图模型层；RxSwift：知乎 iOS 客户端采用了 RxSwift 响应式编程框架，使得代码更加简洁、清晰；Alamofire：知乎 iOS 客户端采用了 Alamofire 网络请求框架，方便网络请求的管理；Kingfisher：知乎 iOS 客户端使用了 Kingfisher 图片加载框架，提高了图片加载的效率和性能。

3) 技术架构

知乎 iOS 客户端采用了 MVVM 架构，将各个功能模块分为模型层、视图层和视图模型层。模型层负责数据的获取和处理，视图层负责展示数据和用户交互，视图模型层负责将模型层的数据转换为视图层可用的数据格式，并将用户的操作反馈回模型层。MVVM 架构可以使代码更加清晰、模块化，便于后续扩展和维护。

4) 功能模块

知乎 iOS 客户端的功能模块如下：

首页：展示热门问题和关注的话题，推荐优秀回答和文章；问题：展示问题的基本信息和回答，用户可以关注问题、点赞或评论回答；话题：展示话题的基本信息和相关问题、回答，用户可以关注话题、提问或回答问题；个人中心：展示用户的个人信息、动态、收藏、提问等信息；消息中心：展示用户的消息、通知等信息。

5) 开发方法

在知乎 iOS 客户端的开发中，采用了敏捷开发的方法。敏捷开发是一种以迭代、循序渐进的方式进行开发的方法。在知乎的开发中，每个开发周期一般为 2 周，每个开发周期结束后会进行一次迭代。在每个迭代中，会完成一些小的功能，或者对已有的功能进行优化和改进。这种方法可以让开发团队更加灵活地应对需求变化，同时也可以让产品更快上线，提高产品的迭代速度和质量。

6) 总结

知乎 iOS 客户端是一个基于 MVVM 架构，采用 Swift/Objective-C、RxSwift、Alamofire、Kingfisher 等技术栈的知识分享应用。该应用为用户提供了丰富的功能和良好的用户体验。在开发中采用了敏捷开发的方法，可以让开发团队更加灵活地应对需求变化，同时也可以让产品更快上线，提高产品的迭代速度和质量。

18.4.4 网页App开发方式

iOS/Android 的内置浏览器是基于 webkit 内核的，很多 App 开发都是用 HTML 或 HTML5、CSS3、JavaScript 技术，服务端技术用 Java、PHP、ASP。相对来说，网页开发涉及的技术点较少。

Web App 指采用 HTML5 缩写的 App，不需要下载安装。类似于现在所说的轻应用。生存于浏览器中的应用，可以说是触屏版的网页应用。Web 应用本质上是为移动浏览器设计的基于 Web 的应用，它们是用普通 Web 开发语言开发的，可以在各种智能手机浏览器上运行。

Web App 开发即是一种框架型 App 开发模式（HTML5 App 框架开发模式），该开发具有跨平台的优势，该模式通常由“HTML5 云网站 + App 应用客户端”两部分构成，App 应用客户端只需安装应用的框架部分，而应用的数据则是每次打开 App 时，去云端取数据呈现给手机用户。

HTML5 应用程序使用标准的 Web 技术，通常是 HTML5、JavaScript 和 CSS。这种只编写一次、可多处运行的移动开发方法构建的跨平台移动应用程序可以在多个设备上运行。虽然开发人员仅使用 HTML5 和 JavaScript 就能构建功能复杂的应用程序，但仍然存在一些重大的局限性，具体包括会话管理、安全离线存储以及访问原生设备功能等。

下面介绍去哪儿网网页 App 案例。

1) 项目背景

该项目是一个名为“去哪儿”的旅游类网页 App，主要功能包括用户注册、登录和个人信息管理、景点查询和推荐、酒店预订和评论、路线规划和导航以及旅游攻略和分享。

在该项目中，用户可以浏览各个城市的景点信息，查看酒店的位置和价格，规划旅游路线，分享旅游经验和照片等。同时，用户可以对酒店和景点进行评论和评分，提供给其他用户参考。在导航功能方面，该项目使用了百度地图 API 来实现定位和导航功能。

该项目在上线后获得了很好的用户反馈和口碑，并且在当地旅游市场上具有一定的知名度。

2) 技术栈

React Native 是 Facebook 公司开源的一款跨平台移动应用开发框架，它可以让开发者使用 JavaScript 和 React 的语法来构建 iOS 和 Android 应用程序。2015 年，Airbnb 公司开始使用 React Native 来开发其移动应用。在 Airbnb 的成功案例中，React Native 开发周期较短，开发效率高，同时能够实现良好的用户体验。因此，越来越多的公司开始使用 React Native 来开发他们的移动应用。

3) 项目开发周期

该项目从 2018 年 1 月开始，历时约 6 个月。其中，前期需求分析与原型设计共计用时约 1 个月，开发周期为 3 个月，测试与上线用时约 2 个月。

4) 项目架构

该项目采用 React Native 开发框架，同时使用 Redux 作为状态管理工具。前端 UI 框架使用了 Ant Design Mobile，后端采用 Node.js 和 Express 框架，数据库使用 MongoDB。在构建过程中，使用了 React Native CLI，代码托管在 GitHub 上，使用了 Travis CI 进行自动化构建和测试。

18.4.5 混合App开发

混合开发是一种结合原生和 HTML5 开发的技术，取长补短的一种开发模式。它主要以 JS+Native 两者相互调用为主，从开发层面实现“一次开发，多处运行”的机制，成为真正适合跨平台的开发。混合应用开发的原生代码部分利用 WebView 插件或者其他的框架为 HTML5 提供了一个容器，程序主要的业务实现、界面展示是利用 H5 相关的 Web 技术进行实现的。

混合开发有以下优点：一次开发，多处运行；兼具了 Native App 良好用户体验的优势，也兼具了 Web App 使用 HTML5 跨平台开发低成本的优势；可以集成第三方 SDK；可以使用原生 API；可以使用 Web 技术。

混合开发也有以下缺点：性能不如原生应用；需要依赖 WebView；难以实现复杂交互效果。

开发人员可以使用 Ionic、React Native、Flutter 等框架来进行混合开发。这些框架都有自己的特点和优势，开发人员可以根据自己的需求选择适合自己的框架。

1. 混合应用开发方法

在 Uber 混合应用程序中，原生代码部分利用了 WebView 插件为 HTML5 提供了一个容器，程序主要的业务实现、界面展示是利用 H5 相关的 Web 技术进行实现的。这种开发方法具有以下优点：

（1）跨平台。混合应用程序可以同时运行在多个操作系统和设备上，如 Android 和 iOS 等，大大提高了应用程序的覆盖面和用户量。

（2）一次开发，多处运行。混合应用程序可以通过“一次开发，多处运行”的方式，减少开发成本和维护成本，提高开发效率。

（3）丰富的 Web 技术支持。混合应用程序可以使用 HTML、CSS、JavaScript 等丰富的 Web 技术，具有良好的可扩展性和可维护性。

（4）原生功能支持。混合应用程序可以利用原生代码部分提供的功能，如调用摄像头、GPS 等硬件设备，提供更好的用户体验。

2. 混合 App 开发案例

1) 开发背景

Uber 是一家美国的科技公司，成立于 2009 年，旨在通过移动互联网技术打造一种全新的出行方式，提供便捷、高效、安全的打车服务。在 Uber 成立初期，公司采用了传统的原生应用程序开发方式，然而随着公司的快速发展和用户量的增加，Uber 在移动应用程序开发上遇到了一些挑战，如不同操作系统的兼容性、应用程序体积过大等问题。为了解决这些问题，Uber 开始尝试采用混合应用开发模式，结合原生和 HTML5 开发技术，实现“一次开发，多处运行”的目标。

2) 开发方法

Uber 采用了混合应用开发模式，即结合原生和 HTML5 开发技术。在该项目中，原生代码部分利用了 WebView 插件为 HTML5 提供了一个容器，程序主要的业务实现、界面展示是利用 H5 相关的 Web 技术进行实现的。Uber 使用了 Ionic 框架来实现混合应用开发，Ionic 是一个基于 AngularJS 和 Cordova 的开源框架，提供了丰富的 UI 组件和工具，适用于移动应用程序的开发。

3) 技术栈

Uber 混合应用程序的技术栈主要包括以下几方面。

Ionic 是一个基于 AngularJS 和 Cordova 的开源框架，提供了丰富的 UI 组件和工具，适用于移动应用程序的开发；AngularJS 是一个由 Google 开发的 JavaScript 框架，适用于 Web 应用程序的开发，具有强大的数据绑定和 MVC 架构支持；Cordova 是一个开源的移动应用程序开发框架，可以将 HTML、CSS 和 JavaScript 代码打包成原生应用程序并在多个平台上运行；Node.js 是一个基于 Chrome V8 引擎的 JavaScript 运行环境，适用于服务器端应用程序的开发；MongoDB 是一个基于文档的 NoSQL 数据库，适用于大数据量、高并发的应用程序。

4) 项目架构

Uber 混合应用程序的项目架构主要分为前端和后端两部分。前端采用 Ionic 框架, 使用 HTML、CSS 和 JavaScript 等 Web 技术进行开发。后端使用 Node.js 和 MongoDB 进行开发, 提供数据存储和业务逻辑处理等功能。在该项目中, 原生代码部分利用了 WebView 插件为 HTML5 提供了一个容器, 程序主要的业务实现、界面展示是利用 H5 相关的 Web 技术进行实现的。

5) 功能模块

Uber 混合应用程序的主要功能模块包括注册、登录和个人信息管理, 打车服务和订单管理, 地图定位和导航服务, 支付和账单管理, 评价和反馈服务。

在该项目中, 用户可以注册、登录并管理个人信息, 使用地图定位和导航服务来请求打车服务, 查看和管理订单, 使用支付功能进行在线支付, 评价和反馈服务等。

6) 总结

Uber 混合应用程序采用了混合应用开发模式, 结合原生和 HTML5 开发技术, 实现“一次开发, 多处运行”的目标。该项目的技术栈主要包括 Ionic 框架、AngularJS、Cordova、Node.js 和 MongoDB 等。在该项目中, 原生代码部分利用了 WebView 插件为 HTML5 提供了一个容器, 程序主要的业务实现、界面展示是利用 H5 相关的 Web 技术进行实现的。

18.5 无代码开发

18.5.1 无代码开发的发展阶段

无代码开发的概念可以追溯到 20 世纪 50 年代的第一代编程语言。随着时间的推移, 技术和工具的发展使得非专业开发人员也能参与软件开发过程。无代码开发的概念于 2014 年左右开始崭露头角, 随着云计算和可视化开发工具的出现, 无代码开发迅速发展, 并在近年来得到了广泛应用。

无代码开发经历了几个发展阶段。第一阶段是单一功能的代码生成工具, 如报表生成工具和表单生成工具; 第二阶段是面向业务的应用开发平台, 如 PowerBuilder 和 Visual Basic 等; 第三阶段是基于模型驱动的开发平台, 如 Mendix 和 OutSystems 等; 第四阶段是基于低代码开发平台和云计算的无代码开发平台, 如 Salesforce 和 Airtable 等。

无代码开发的应用领域非常广泛, 包括企业级应用、移动应用、网站开发、物联网和人工智能等领域。无代码开发在这些领域中可以提高开发效率, 降低开发成本, 并且可以使非专业人员也能够参与开发。

据市场研究公司 Gartner 的报告显示, 到 2024 年, 无代码开发平台市场规模将达到 27 亿美元。而且随着越来越多的企业和组织开始使用无代码开发平台, 无代码开发市场的发展前景非常广阔。

18.5.2 无代码开发的优劣

无代码开发的主要优点是可以降低开发应用程序的技术门槛, 使得非专业开发人员也可以

快速构建应用程序。此外，无代码开发还可以提高开发效率，减少开发成本，缩短开发周期。对于企业来说，无代码开发可以使其更快速地推出新产品或服务，提高业务效率和竞争力。

然而，无代码开发也存在一些缺点。首先，无代码开发的灵活性和可扩展性有限，无法满足一些特殊的需求。其次，由于开发者无须编写代码，可能会导致应用程序的性能和安全性问题。最后，无代码开发的开发成本可能会高于传统的编码开发方式，因为开发者需要购买和使用无代码平台。

加速应用开发：无代码开发平台提供了可视化的界面和组件，使开发过程更加快速和简单。无须编写代码，开发人员可以通过拖放组件、配置属性和规则来创建应用程序。

降低技术门槛：无代码开发使非专业开发人员能够参与软件开发。无须精通编程语言和复杂的技术知识，任何人都可以通过简单的操作创建应用程序。

可扩展性和可维护性：无代码开发平台提供了模块化和可重用的组件，使应用程序更易于扩展和维护。开发人员可以通过组合和定制现有组件来构建复杂的应用程序。

限制自定义和灵活性：无代码开发平台通常提供了一系列预定义的组件和功能，可能无法满足某些特定需求。开发人员可能会受到平台限制，无法进行高度定制或实现特定的复杂功能。

依赖第三方平台：无代码开发平台通常是基于特定的工具和框架构建的，开发人员需要依赖这些平台来创建和运行应用程序。这可能增加了一定的依赖性和风险。

18.5.3 主流开发平台和测试平台

主流无代码开发平台包括 Airtable、Bubble、OutSystems、Mendix、AppSheet、Salesforce Lightning Platform 等。这些平台提供了可视化的界面、组件库和自定义规则，使开发人员能够轻松创建应用程序而无须编写代码。

1. 主流低代码开发平台

目前，市场上存在许多无代码开发和低代码开发平台，以下是其中一些主流平台：

(1) Airtable。一款基于云的电子表格和数据库平台，可以帮助用户轻松创建和管理数据。

(2) Bubble。一款可视化编程平台，允许开发者通过拖放组件来创建 Web 应用程序。

(3) Microsoft PowerApps。微软提供的一款低代码开发平台，可以帮助用户快速构建企业级应用程序。

(4) Google App Maker。一款基于云的低代码开发平台，允许开发者轻松创建自定义应用程序。

2. 主流无代码测试平台

在无代码开发中，测试平台也是非常重要的。以下是一些主流的测试平台：

(1) TestComplete。一款功能强大的自动化测试工具，可以帮助开发者轻松创建和运行测试脚本。

(2) Ranorex Studio。一款易于使用的自动化测试工具，可以帮助开发者快速创建和运行测试脚本。

18.5.4 无代码开发流程

1. 无代码开发流程

无代码开发流程具体包括：

- (1) 需求分析。确定应用的目标、功能和性能需求。
- (2) 界面设计。使用可视化编辑器创建界面布局和样式。
- (3) 逻辑实现。通过拖放组件、配置属性等方式实现功能逻辑。
- (4) 数据管理。设置数据库结构、API 调用等数据交互方式。
- (5) 测试与调优。对应用进行功能测试、性能优化等工作。
- (6) 发布与维护。将应用发布到目标平台，进行后期维护和更新。

2. 无代码开发案例

无代码开发已经得到了越来越广泛的应用，下面是一些成功的无代码开发案例：

(1) 联合国世界粮食计划署的“数字家庭食品援助计划”。该项目旨在通过无代码开发平台来帮助难民和贫困家庭获得食品援助，使用无代码开发平台 Quick Base，创建了一个应用程序来跟踪和管理食品援助的分配和分发。

(2) 沃尔玛的 OneOps 平台。沃尔玛使用无代码开发平台 OneOps 来创建和管理其云基础设施。使用该平台，沃尔玛可以快速创建和部署应用程序，并更轻松地进行容量规划和资源管理。

(3) 布鲁克林公共图书馆的 BookMatch 应用程序。该应用程序是一个无代码开发的图书推荐平台，为用户提供了个性化的书籍推荐服务；使用无代码开发平台 Bubble 进行开发，使开发团队能够快速创建和部署应用程序。

(4) 雀巢的 Nespresso 应用程序。该应用程序是一个无代码开发的电商平台，为用户提供了购买咖啡胶囊的便捷服务；使用无代码开发平台 Mendix 进行开发，使开发团队能够快速创建和部署应用程序。

(5) 新加坡政府的业务快车平台。该平台是一个无代码开发的应用程序，旨在帮助企业更快速地获得政府服务；使用无代码开发平台 OutSystems 进行开发，使开发团队能够快速创建和部署应用程序。

这些无代码开发的成功案例证明了无代码开发的潜力和价值，可以帮助企业更快速地构建应用程序，提高业务效率和竞争力。

18.5.5 无代码开发案例：Zapier

1. 项目背景

Zapier 于 2011 年由 Wade Foster、Mike Knoop 和 Bryan Helmig 三名校友共同创立。他们认为软件应该连接人与人，而不是孤立存在。因此，Zapier 致力于将各种应用程序集成在一起，让人们可以轻松地自动完成任务。Zapier 成立于 2011 年，至今已有 10 多年的历史。

2. 开发方法

Zapier 采用无代码开发方法，用户通过可视化拖曳的方式创建自动化流程，无须编写任何

代码。具体的开发流程如下：

（1）选择触发器。选择一个“触发器应用”和一个事件，当事件被触发时，自动化流程开始运行。例如，选择“每次收到新电子邮件”这个触发器。

（2）选择动作。选择想要触发的“动作应用”和想要运行的动作。例如，选择“将新邮件添加到 Trello 看板”这个动作。

（3）设置触发器和动作之间的映射关系。选择连接触发器事件和动作之间的参数。例如，选择将“邮件主题”作为 Trello 的“卡片标题”。

（4）命名和激活 Zap。为 Zap 命名，然后激活它。一旦激活，Zap 就会开始自动运行。每当触发器事件发生时，Zap 就会执行相应的动作。

（5）管理和修改。随时可以停用、修改或删除 Zap，也可以查看 Zap 的运行活动和历史记录详细信息。

（6）添加订阅。如果需要，可以添加高级订阅以执行更多的动作或更频繁的 Zap。

（7）Zapier 开放平台。Zapier 有一个开放平台，允许第三方开发者创建自己的 Zap 模板与 Zapier 的用户共享。

3. 项目模块功能

项目模块功能具体包括：

（1）触发器。选择触发器应用和事件，Zap 运行的起点。

（2）动作。选择动作应用和动作，事件触发时会自动执行。

（3）搜索。搜索 Zap 模板以快速创建 Zap。

（4）连接器。3000 多个应用程序的连接器，支持各种应用集成。

（5）Zap。通过拖曳触发器、动作和映射关系创建的自动化流程。

（6）模板。Zapier 开放平台上的 Zap 模板，方便用户快速使用。

（7）订阅。添加高级订阅以获得更多 Zap 功能。

（8）分析。查看 Zap 运行数据，优化 Zap。

通过以上开发流程，Zapier 实现了 3000 多个应用程序之间的自动集成与工作流自动化，大大提高了用户的工作效率。这也使其成为无代码开发的典型案例。